Binary Tree Data Structure

What is a Binary Tree?

A binary tree is a hierarchical data structure where each node has at most two children, referred to as the left child and right child. The topmost node is called the root, and nodes with no children are called leaves.

Basic Structure of a Binary Tree Node

```java
class TreeNode {
    int data;
    TreeNode left;
    TreeNode right;

    // Constructor
    public TreeNode(int data) {
        this.data = data;
        this.left = null;
        this.right = null;
    }
}
```

Binary Tree Traversals

Tree traversal refers to the process of visiting each node in the tree exactly once in a systematic way. There are four main types of traversals:

1. PreOrder Traversal (Root → Left → Right)

In preorder traversal, we visit the root node first, then recursively traverse the left subtree, followed by the right subtree.

Algorithm:

1. Visit the root node
2. Traverse the left subtree
3. Traverse the right subtree

```java
public void preOrderTraversal(TreeNode root) {
    if (root == null) {
        return;
    }

    // Visit root
    System.out.print(root.data + " ");

    // Traverse left subtree
    preOrderTraversal(root.left);

    // Traverse right subtree
    preOrderTraversal(root.right);
}
```

Example: For a tree with structure:

```
    1
   / \
  2   3
 / \
4   5
```

PreOrder output: 1 2 4 5 3

2. InOrder Traversal (Left → Root → Right)

In inorder traversal, we recursively traverse the left subtree first, then visit the root node, and finally traverse the right subtree.

Algorithm:

1. Traverse the left subtree
2. Visit the root node
3. Traverse the right subtree

```java
public void inOrderTraversal(TreeNode root) {
    if (root == null) {
        return;
    }

    // Traverse left subtree
    inOrderTraversal(root.left);

    // Visit root
    System.out.print(root.data + " ");

    // Traverse right subtree
    inOrderTraversal(root.right);
}
```

Example: For the same tree:

```
     1
    / \
   2   3
  / \
 4   5
```

InOrder output: 4 2 5 1 3

3. PostOrder Traversal (Left → Right → Root)

In postorder traversal, we recursively traverse the left subtree first, then the right subtree, and finally visit the root node.

Algorithm:

1. Traverse the left subtree
2. Traverse the right subtree
3. Visit the root node

```java
public void postOrderTraversal(TreeNode root) {
    if (root == null) {
        return;
    }

    // Traverse left subtree
    postOrderTraversal(root.left);

    // Traverse right subtree
    postOrderTraversal(root.right);

    // Visit root
    System.out.print(root.data + " ");
}
```

Example: For the same tree:

```
    1
   / \
  2   3
 / \
4   5
```

PostOrder output: 4 5 2 3 1

4. LevelOrder Traversal (Breadth-First Search)

Level order traversal visits nodes level by level from left to right. This traversal uses a queue data structure to keep track of nodes to visit.

Algorithm:

1. Start with the root node in a queue
2. While queue is not empty:
    - Remove a node from the front of the queue
    - Visit the node
    - Add its left child to the queue (if exists)
    - Add its right child to the queue (if exists)

```java
import java.util.Queue;
import java.util.LinkedList;

public void levelOrderTraversal(TreeNode root) {
    if (root == null) {
        return;
    }

    Queue<TreeNode> queue = new LinkedList<>();
    queue.offer(root);

    while (!queue.isEmpty()) {
        TreeNode current = queue.poll();
        System.out.print(current.data + " ");

        if (current.left != null) {
            queue.offer(current.left);
        }

        if (current.right != null) {
            queue.offer(current.right);
        }
    }
}
```

Example: For the same tree:

```
     1
    / \
   2   3
  / \
 4   5
```

LevelOrder output: 1 2 3 4 5

Complete Binary Tree Implementation

java

```java
import java.util.Queue;
import java.util.LinkedList;

class BinaryTree {
    TreeNode root;

    public BinaryTree() {
        root = null;
    }

    // PreOrder Traversal
    public void preOrder(TreeNode node) {
        if (node == null) return;
        System.out.print(node.data + " ");
        preOrder(node.left);
        preOrder(node.right);
    }

    // InOrder Traversal
    public void inOrder(TreeNode node) {
        if (node == null) return;
        inOrder(node.left);
        System.out.print(node.data + " ");
        inOrder(node.right);
    }

    // PostOrder Traversal
    public void postOrder(TreeNode node) {
        if (node == null) return;
        postOrder(node.left);
        postOrder(node.right);
        System.out.print(node.data + " ");
    }

    // LevelOrder Traversal
    public void levelOrder(TreeNode root) {
        if (root == null) return;
        Queue<TreeNode> queue = new LinkedList<>();
        queue.offer(root);

        while (!queue.isEmpty()) {
            TreeNode current = queue.poll();
            System.out.print(current.data + " ");
```

```
            if (current.left != null) queue.offer(current.left);
            if (current.right != null) queue.offer(current.right);
        }
    }
}
```

Quiz Questions

Question 1: What is the correct order of visiting nodes in PreOrder traversal? a) Left → Root → Right b) Root → Left → Right c) Left → Right → Root d) Right → Root → Left

Answer: b) Root → Left → Right

Question 2: For a binary tree with root value 10, left child 5, and right child 15, what will be the InOrder traversal output? a) 10 5 15 b) 5 10 15 c) 5 15 10 d) 15 10 5

Answer: b) 5 10 15

Question 3: Which traversal method uses a queue data structure? a) PreOrder b) InOrder c) PostOrder d) LevelOrder

Answer: d) LevelOrder

Question 4: In PostOrder traversal, when is the root node visited? a) First b) Second c) Last d) It depends on the tree structure

Answer: c) Last

Question 5: What will be the PostOrder traversal of a tree with structure: Root(1), Left child(2), Right child(3)? a) 1 2 3 b) 2 1 3 c) 2 3 1 d) 3 2 1

Answer: c) 2 3 1

Practice Coding Questions

Question 1: Write a recursive method to count the total number of nodes in a binary tree.

Question 2: Implement a method to find the height (or depth) of a binary tree using recursion.

Question 3: Create a method that returns true if two binary trees are identical (same structure and same node values), false otherwise.

Question 4: Write a recursive function to find the maximum value in a binary tree.

Question 5: Implement a method to check if a binary tree is a valid Binary Search Tree (BST) where left child values are less than parent and right child values are greater than parent.